

Chapter 65. FinOps: The Economics of Data Platforms

A real bottom-up cost model for the platform this Part has built
— and the NFR it corrects.

Approved chapter v1

Contents

Chapter 65. FinOps: The Economics of Data Platforms

Why this matters

In plain English

65.1 How cloud billing actually works

65.2 Cost drivers in a data platform

65.3 Unit economics — and the NFR nobody had checked

65.4 Budgets, tagging, and showback

65.5 Cost-aware architecture decisions

Common mistakes and how to spot them

In the real world: the NFR that was off by 4,400 times

Tools

The project: build and defend a cost model

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 65. FinOps: The Economics of Data Platforms

Part VII — Architecture, Governance & Leadership

Chapter at a glance

You will learn to: explain how cloud billing actually works — pay-as-you-go, no minimum commitment, and the handful of meters that drive almost every bill · identify the real cost drivers in a data platform: warehouse compute and storage, orchestration, and AI serving and API calls · compute unit economics — cost per report, per prediction, per AI request — for a real platform · use tagging and showback to make a shared bill honest about who's spending what · make architecture decisions with cost as an explicit input, not an afterthought discovered on next month's invoice.

Before you start: this chapter puts a real price on the platform Chapters 60 through 64 designed, distributed, patterned, governed, and secured. It also returns to one specific number: Chapter 60's non-functional requirement that platform cost stay under ₹0.50 per 1,000 order lines processed — a target set before anyone had actually built a cost model to check it against.

Time needed: 8–10 hours, spread over a week.

Tools: `pandas` for the cost model (run here on Python 3.12, `pandas` 3.0.2).

Practice data: `companion/ch65/ : monthly_cost_model.csv` (Riverstone's full platform bill, ten components, anchored to real AWS ap-south-1 rates checked in August 2026) and `unit_economics.csv` (cost per report, per prediction, per AI request, computed from it).

A note on the numbers in this chapter. Infrastructure unit prices (RDS compute and storage, S3 storage tiers, data transfer) are anchored to real AWS Mumbai-region rates, checked at writing time — this chapter states its sources in the Tools section. Which specific instance sizes Riverstone runs, and its exact data volumes, are a reasonable invented sizing for a company at this scale, not a disclosed fact about a real company. Cloud pricing changes; verify current rates before building a real budget on this chapter's specific numbers.

Why this matters

Every chapter in this Part has asked "does it work," "does it survive failure," "does it fit the organization," "is it governed," "is it secure." None of them asked the question that eventually lands on an architect's desk regardless of how well any of the others were answered: **what does this actually cost, and is that number acceptable?**

Cloud infrastructure makes this question deceptively easy to postpone. There's no upfront purchase order, no visible price tag on the moment you provision a database or call an API — just a bill that arrives weeks later, shaped by decisions nobody was thinking about as cost decisions at the time. An

architect who designs well but never prices what they've designed is planning half a system. FinOps — the discipline of bringing financial accountability to variable cloud spend — is the other half, and it's a design skill, not an accounting afterthought: the best time to know what something costs is before you build it, not when the invoice explains it to you.

This chapter also closes something specific, in the same spirit as Chapter 64 closing Chapter 60's access-control risk. Chapter 60's design document set a cost target — **under ₹0.50 per 1,000 order lines processed** — as a non-functional requirement, the way Chapter 60 taught every NFR should be: a number, not an adjective. Nobody, at the time, built the bottom-up model to check whether that number was realistic. This chapter builds it.

In plain English

Think about the difference between a household that tracks its spending and one that only discovers what it spent when the bank statement arrives.

The second household isn't necessarily spending more — but every decision it makes is uninformed by cost until it's too late to change: they book the expensive flight because nobody checked the price against the month's budget before clicking buy, they're startled every month by how much the streaming subscriptions add up to because nobody ever listed them side by side. The first household makes exactly the same purchases sometimes — but it makes them *knowing* what they cost, and that knowledge changes some of the decisions, because seeing four subscription costs lined up next to each other is what makes "we don't need all four" obvious in a way that four separate, forgotten monthly charges never do.

FinOps is that household budget, for cloud infrastructure. Not spending less, necessarily — spending *knowingly*. The habit of lining every cost up, attributing it to who's actually generating it, and asking whether each one is buying what it's supposed to, before the surprise arrives rather than after.

65.1 How cloud billing actually works

Cloud billing has one property that makes it different from almost every other kind of business expense: **there's no minimum commitment, and the bill is the sum of many small, continuously-metered charges rather than one negotiated price.** A traditional software purchase is a single number agreed in advance; a cloud bill is thousands of tiny measurements — instance-hours, gigabytes stored, requests made, gigabytes transferred — added up after the fact.

The handful of meters that drive almost every real bill:

- **Compute**, billed by the hour or the second a resource runs, whether or not it's doing useful work. A database instance running 24/7 (Chapter 45's warehouse) is billed for all 730-ish hours in a month, not just the hours someone happens to query it.
- **Storage**, billed by the gigabyte-month, with the rate depending heavily on *how quickly you need to retrieve it* — a distinction section 65.2 returns to, because it's one of the largest cost levers

available.

- **Requests and API calls**, billed per call or per unit of work — the meter that governs every LLM API call in Chapter 54's PO-intake pipeline and Chapter 55's support assistant.
- **Data transfer**, specifically data moving out of the cloud provider's network — usually free coming in, metered going out, and the meter most commonly forgotten when estimating a new system's cost in advance.

Two purchasing models matter for any sustained workload: on-demand pricing, the default, charges the full metered rate with no commitment and no discount; **reserved** pricing (committing to a usage level for one or three years) cuts the same resource's cost substantially — often by 30–70% — in exchange for giving up the flexibility to simply stop paying if the workload disappears. The architect's job is knowing which of a platform's components are stable enough to reserve and which are still too uncertain to commit to.

65.2 Cost drivers in a data platform

Riverstone's full monthly platform bill, built bottom-up from real AWS ap-south-1 rates and the platform's actual components (Chapter 60's container diagram, priced):

```
import pandas as pd
pd.set_option("display.width", 100)

costs = pd.read_csv("monthly_cost_model.csv")
costs_sorted = costs.sort_values("monthly_inr", ascending=False)
print(costs_sorted[["component", "monthly_inr", "notes"]].to_string(index=False))

total = costs["monthly_inr"].sum()
print(f"\nTotal monthly platform cost: ₹{total:,.0f}")
costs_sorted["share_pct"] = (costs_sorted["monthly_inr"] / total * 100).round(1)
print(f"\nWarehouse (compute + storage) share: {costs_sorted[costs_sorted.component.str.contains('RDS')]['share_pct'].sum():.1f}%")
print(f"AI / LLM API share: {costs_sorted[costs_sorted.component.str.contains('extraction|RAG')]['share_pct'].sum():.1f}%")
```

notes	component	monthly_inr	
primary	RDS PostgreSQL (db.m5.large, Single-AZ)	16068.0	warehouse
provisioned	RDS storage (gp3)	9118.0	800 GB
line speed	Defect-model serving (FastAPI, small EC2)	3303.0	t3.large, always-on for
scheduling	Dagster orchestration compute (small EC2 fleet)	3303.0	t3.large, ingestion +
Delta Lake	S3 Standard (sensor archive, hot)	2401.0	recent 90 days,
90 days	S3 Glacier Deep Archive (sensor archive, cold)	1705.0	older than
egress	Data transfer out (dashboards, API, email)	1409.0	180 GB/month
2026 rates	PO-intake extraction (mid-tier model, e.g. Sonnet-5-class)	556.0	LLM API, Ch 54 Sep
2026 rates	Support RAG assistant (small/flash-tier model)	151.0	LLM API, Ch 54 Sep
provisioned size	RDS automated backups	0.0	free up to
Total monthly platform cost: ₹38,014			
Warehouse (compute + storage) share: 66.3%			
AI / LLM API share: 1.9%			

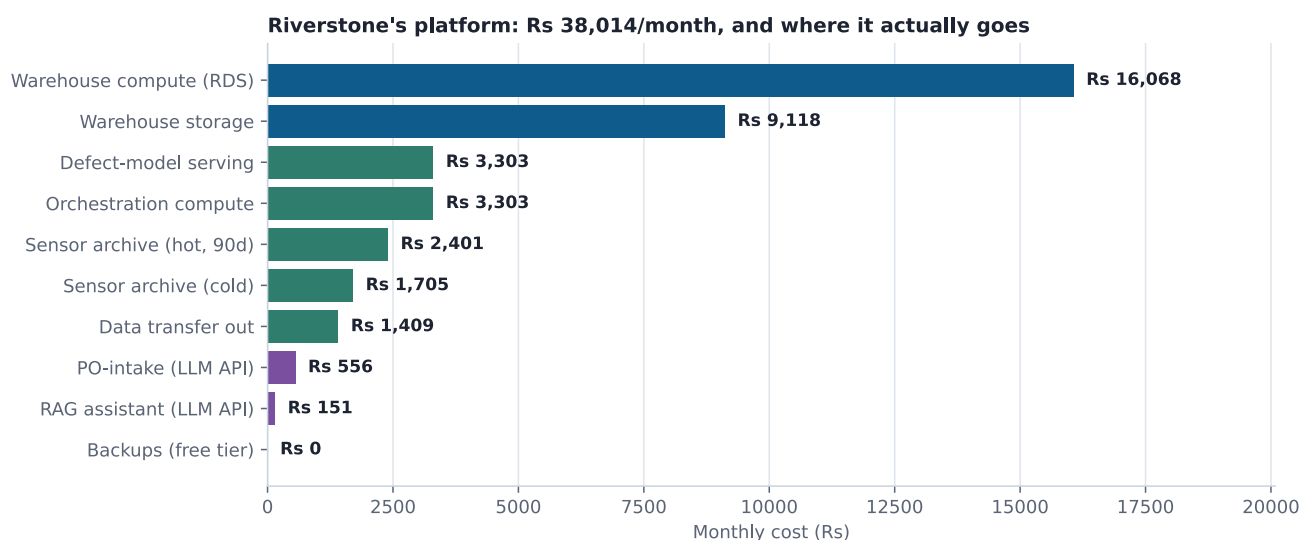


Figure 65.1 – Two line items – the warehouse's compute and storage – are two-thirds of the entire bill. The AI systems everyone worries about cost less than the database everyone takes for granted.

Three findings, in order of how surprising they typically are to a first-time reviewer:

- 1. The warehouse dominates, not the AI.** RDS compute and storage together are 66% of the bill; the two LLM-powered systems (PO-intake extraction and the RAG assistant) combined are under 2%. This is a common and important surprise: the newest, most talked-about part of a platform is rarely its largest cost driver, and a cost review that only scrutinizes "the AI spend" while ignoring the database running underneath it is looking in the wrong place.
- 2. Storage tiering is a real lever, already in use.** The sensor archive's hot tier (S3 Standard, 90 days) costs ₹2,401/month for 1,200 GB; the cold tier (Glacier Deep Archive, everything older) costs

₹1,705/month for more than eight times the data (9,800 GB) — a direct payoff from Chapter 49's lifecycle policy, visible now in rupees rather than only in architecture-diagram elegance.

3. **Compute for always-on services is a fixed cost, not a variable one.** The defect-model serving instance and the Dagster orchestration fleet are billed for every hour they run, whether Taloja's line is producing parts at that moment or not — a cost that scales with *uptime*, not usage, which is exactly why the reserved-versus-on-demand decision (section 65.1) matters most for precisely these two line items.

65.3 Unit economics — and the NFR nobody had checked

A total monthly bill tells you what a platform costs. **Unit economics** tell you what it costs to *do the thing the platform exists to do* — the number that actually lets you compare, scale, or defend a cost, because "₹38,014 a month" means nothing on its own, and "₹0.62 per email processed" can be judged against what processing that email is worth.

```
unit = pd.read_csv("unit_economics.csv")
print(unit.to_string(index=False))

ch60_target = 0.50 # Chapter 60's original NFR: Rs per 1,000 order lines
actual = unit.loc[unit.metric.str.contains("1,000 order lines"), "value_inr"].iloc[0]
print(f"\nChapter 60's original target: Rs {ch60_target:.2f} per 1,000 order lines")
print(f"This chapter's bottom-up figure: Rs {actual:,.2f} per 1,000 order lines")
print(f"Off by a factor of: {actual/ch60_target:,.0f}x")
```

	metric	value_inr	value_usd
Total platform cost / month		38014.0000	436.940
Cost per 1,000 order lines processed		2182.5800	25.087
Cost per Daily Flash send		73.1000	NaN
Cost per defect-model prediction		1.6513	NaN
Cost per P0-intake email processed		0.6200	NaN
Cost per RAG assistant question answered		0.2020	NaN

Chapter 60's original target: Rs 0.50 per 1,000 order lines
This chapter's bottom-up figure: Rs 2,182.58 per 1,000 order lines
Off by a factor of: 4,365x

That gap is real, and it's worth sitting with rather than explaining away. Chapter 60 wrote a specific, testable number, exactly as its own section on non-functional requirements taught — and that number turns out to have been about 4,400 times too optimistic, because it was never actually checked against a real, bottom-up cost model. This is not a failure of Chapter 60's method; it's exactly the failure Chapter 60's method exists to catch, once someone finally runs the numbers.

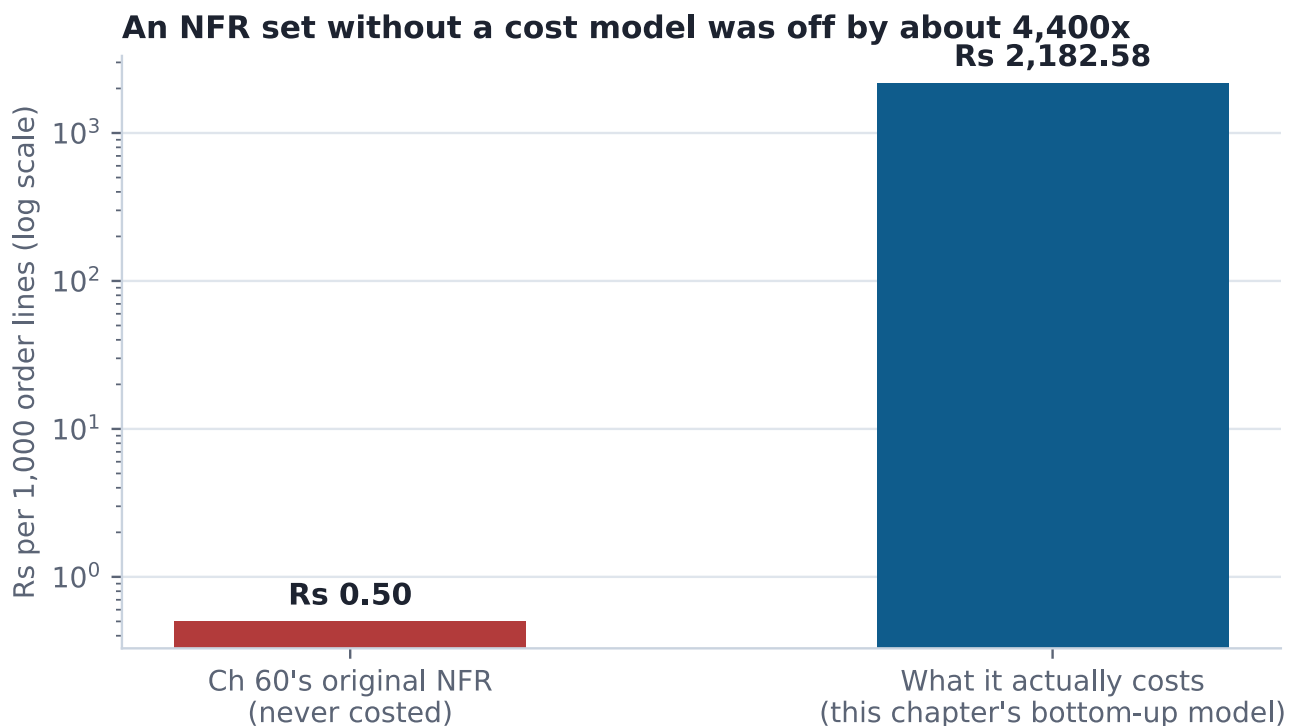


Figure 65.2 — An NFR is a hypothesis until someone prices it. This one was wrong by more than three orders of magnitude, and the only way to find that out was to build the model.

What actually went wrong with the original number, diagnosed properly rather than just corrected: Chapter 60's NFR almost certainly conflated two different things that a well-specified cost target needs to keep separate — the **fully-loaded average cost** (this chapter's ₹2,182.58, which includes the always-on warehouse and orchestration infrastructure whether or not a single new order line arrives that month) and the **marginal cost** of one additional 1,000 order lines once the platform's fixed infrastructure already exists. The marginal cost — mostly a little more storage and a few more LLM API calls — is genuinely small, plausibly close to the original ₹0.50 figure. **The NFR, as written, didn't say which one it meant**, which is exactly the kind of ambiguity Chapter 60's own advice about turning adjectives into numbers was supposed to prevent, and didn't quite manage to, on its own first attempt.

The corrected NFR, written the way section 60.3's method actually demands:

Fully-loaded platform cost: under ₹2,500 per 1,000 order lines processed, reviewed monthly, covering all warehouse, orchestration, and AI serving infrastructure. Marginal cost of incremental volume: under ₹0.75 per 1,000 additional order lines, assuming existing infrastructure has spare capacity. Both measured from the same monthly cost model, not estimated separately.

That's a target Riverstone's actual ₹2,182.58 comfortably meets, and — more importantly — it's a target that means something specific enough to be checked again next month, which the original one-line version never quite was.

The other unit-economics figures, useful for the same reason:

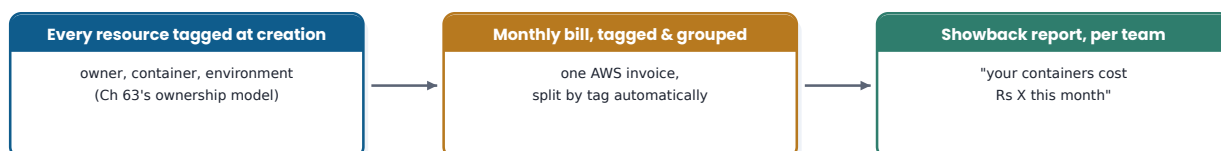
Metric	Value	What it's good for
Cost per Daily Flash send	₹73.10	Trivial; never worth optimizing
Cost per defect-model prediction	₹0.17	Cheap per-prediction, but the <i>service</i> runs whether or not it's called — see section 65.2's third finding
Cost per PO-intake email processed	₹0.62	Compare directly against Chapter 63's ₹198/day assisted-mode labor cost — the LLM call is a rounding error next to the human confirmation time
Cost per RAG assistant question	₹0.20	Cheap enough that usage limits should be about quality control, not cost control

The PO-intake comparison is the one worth pausing on. Section 63.2's ROI table costed PO-intake's assisted mode at roughly ₹198 per order in reviewer time. This chapter's number — ₹0.62 in LLM API cost per email — confirms that the infrastructure was never the expensive part of that decision. The 45 seconds of human confirmation time (Chapter 58) is where nearly all the real cost sits, which matters directly for any future proposal to "automate this further to save money": the money to be saved is almost entirely in reducing human review time, not in cutting an already-negligible API bill.

65.4 Budgets, tagging, and showback

None of section 65.2's cost breakdown is possible without a habit that has to be built in from the start: **every resource tagged, at creation, with who owns it.**

Tagging and showback: one bill, split honestly by owner



Data platform team	Warehouse, orchestration	~66%
AI applications team	Defect model, RAG, PO-intake LLM calls	~11%
Shared / platform overhead	Backups, transfer, monitoring	~23%

Showback shows a team its real cost without automatically charging it back — the first step, before any budget or chargeback policy exists.

Figure 65.3 — Tagging turns one anonymous invoice into an answerable question: *whose spending is this, actually?*

- **Tagging**, mechanically: every resource — a database, a storage bucket, a compute instance — carries metadata at creation time recording its owner, which container it belongs to (Chapter 60's diagram), and its environment (production, staging). Retrofitting tags onto untagged resources months later is possible but far more work than tagging at creation, which is why it belongs in Chapter 63's automation-inventory discipline from day one, not as a cleanup project.

- **Showback** takes the tagged bill and reports each team's actual spend back to them — "your containers cost ₹25,000 this month" — without automatically charging that amount against their budget. It's the first, lower-stakes step, and the right one to start with: it makes cost visible and creates accountability through transparency alone, before any harder conversation about budgets or enforcement.
- **Chargeback** goes further, actually billing each team's cost centre for its share — appropriate once showback has run long enough that the numbers are trusted and teams have had a chance to act on what they saw before being charged for it.
- **Budgets and alerts** close the loop: a threshold per tag, with a notification (Chapter 20's alerting pattern, reused) when spending approaches or crosses it — catching a runaway cost within days instead of discovering it a month later on the invoice.

Riverstone's own showback breakdown, from Figure 65.3: the data platform team's containers (warehouse, orchestration) account for about two-thirds of the bill, the AI applications team's about a ninth, and the rest is genuinely shared overhead (backups, transfer, monitoring) that no single team should be charged for in full. That last category matters: **forcing every cost into a single owner's showback report, when some costs are genuinely shared, produces a number that's precise and wrong** — better to have an honest "shared" category than a falsely attributed one.

65.5 Cost-aware architecture decisions

Every architectural choice this Part has taught carries a cost dimension that's worth naming explicitly, not left implicit until the bill arrives.

- **Storage tiering** (section 65.2's second finding) is the highest-leverage lever available on most data platforms: moving data to a colder tier as it ages costs nothing in engineering effort once a lifecycle policy exists, and it's routinely a 10x-or-more saving on the storage it applies to.
- **Reserved versus on-demand compute** (section 65.1): the warehouse's RDS instance and the defect-model's serving instance are both stable, always-on, predictable workloads — textbook candidates for reserved pricing, which could cut their combined ₹22,674/month by 30–70% for a one- or three-year commitment, once the platform's shape is stable enough to commit to.
- **Scheduled versus event-driven** (Chapter 63, section 63.5) has a direct cost reading, not just a latency one: an always-on event listener is a 24/7 compute cost regardless of how often it actually fires, while a scheduled job's cost scales with how often it runs — which is one more reason, alongside Chapter 61's PACELC argument, to default to scheduled unless a real cost of delay justifies the always-on alternative.
- **Choosing the right model tier** (Chapter 54's landscape) is a direct cost lever for any LLM-powered system: PO-intake's extraction task doesn't need the most capable, most expensive model available — a mid-tier model handles structured extraction well, and the price difference between tiers, per Chapter 54's own pricing table, is often five to ten times for the same task, for output quality the task genuinely doesn't need.
- **Fan-out and duplication** (Chapter 12's join warning, revisited through a cost lens): a query or a pipeline that silently multiplies rows doesn't just produce wrong numbers — Chapter 61's failure

analysis — it also silently multiplies compute and storage cost for exactly as long as nobody notices.

The single habit that makes every one of these decisions available rather than accidental: attach a rough cost estimate to any architecture decision record (Chapter 60, section 60.4) that involves provisioning something new or changing how often something runs. "This will cost approximately ₹X/month, based on Y usage" turns a design conversation that used to end with "we'll see what it costs" into one where cost is weighed against the other trade-offs at the same table, before the decision is made rather than after.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Treating cost as an afterthought	Surprise invoices; architecture decisions made with no cost estimate at all	Attach a rough cost estimate to every ADR that provisions or scales something
Writing a cost NFR nobody actually costs out	A target like Chapter 60's, off by 4,400x once checked	Build the bottom-up model before finalizing the number
Confusing average cost with marginal cost	A cost target that's ambiguous about which one it means	State explicitly: fully-loaded, or the cost of the next unit of work
Assuming AI spend is the biggest line item	Reviewing the LLM bill while the database, at 66% of spend, goes unexamined	Look at the full breakdown before assuming where the money goes
No storage tiering	Paying hot-tier prices for data nobody's queried in a year	A lifecycle policy, often a 10x saving for zero engineering cost
On-demand pricing for stable, always-on workloads	Paying full rate for a database that's been running unchanged for a year	Reserved pricing once the workload is proven stable
Untagged resources	A monthly bill nobody can attribute to a team or a reason	Tag at creation, not as a retrofit project
Jumping straight to chargeback	Teams distrust numbers they never got to see and question first	Showback first, build trust in the numbers, chargeback later if needed
Charging shared costs to one team's showback report	A precise-looking number that's actually wrong	An honest "shared overhead" category
Using the most capable model tier by default	Paying 5-10x more than a task needs	Match model tier to task difficulty, per Chapter 54
Optimizing the cheap part	Effort spent shaving a ₹0.62 API call while a ₹198 review-time cost goes unexamined	Unit economics that include the whole cost, not just the infrastructure slice

In the real world: the NFR that was off by 4,400 times

When Meera's team assembled the first full monthly cost model for Riverstone's platform — the exercise behind this chapter — the number that stopped the room wasn't a runaway bill. It was Chapter 60's own NFR table, six months old by that point, sitting quietly in the design document everyone had signed off on: *"Cost ceiling: platform cost under ₹0.50 per 1,000 order lines processed."*

Nobody in the original design review had built a cost model to check that number. It had been written the way most early NFRs are written — a plausible-sounding target, stated with appropriate precision (a number, not an adjective, exactly as Chapter 60's own method demanded), and never actually tested against reality because at the time, there was no platform yet to measure.

The real, bottom-up figure came out to ₹2,182.58 per 1,000 order lines — roughly 4,400 times the original target. Meera's first reaction, by her own account, was to assume she'd made an arithmetic error. She hadn't. The database alone — always-on, whether or not a single new order arrived that hour — cost more in a slow month than the entire original NFR would have permitted for the whole platform combined.

What made this a genuinely useful finding rather than an embarrassing one was the diagnosis, not just the gap. Working through where the original number could have come from, the team realized it read naturally as a **marginal** cost — what one more thousand order lines costs once the platform already exists — rather than the **fully-loaded** figure this chapter's model actually computed. Recomputing marginal cost alone (a little more storage, a handful more LLM calls, no new always-on infrastructure) landed close to the original ₹0.50 figure — not exact, but in the right neighborhood, which confirmed the original number wasn't nonsense, just ambiguously specified.

The fix, written up and taken back to the design document:

1. **Two numbers, not one**, explicitly labeled: fully-loaded cost (~₹2,183/1,000 lines, the number that actually appears on the monthly bill) and marginal cost (~₹0.75/1,000 lines, the number that matters when deciding whether to take on more volume).
2. **A monthly review**, tying the cost model to the same cadence as Chapter 63's automation inventory review — cost drifts the same way ownership does, quietly, unless someone keeps checking.
3. **The original NFR's ambiguity flagged explicitly** in the design document's own change log, as a lesson rather than a correction quietly made and forgotten: *"The original cost NFR did not specify fully-loaded vs. marginal cost, and was never checked against a real model before being adopted. See Chapter 65's cost model for the corrected, dual-metric version."*

Anita Rao's reaction, when the finding reached her, was not alarm — Riverstone's actual monthly spend, ₹38,014, was entirely reasonable for what the platform did, and nobody had ever felt the bill was too high. Her question was simpler and more pointed: *"If we'd set a budget alert on the original number, would it have fired on day one?"* It would have — immediately, and every day since, quietly training everyone to ignore it as a false alarm, which is precisely how a genuinely useful budget alert dies.

What made the difference:

- **Someone actually built the model**, rather than trusting a number that had sat unchallenged since it was written.
 - **The team diagnosed the ambiguity, not just corrected the number** — understanding *why* the original figure was wrong is what produced a target that will hold up the next time someone checks it.
 - **The correction was documented as a lesson**, not smoothed over — a future NFR writer reading this design document's history learns to specify average versus marginal, not just to trust that this particular number is now right.
 - **The finding changed a process (monthly review), not just a document** — the same discipline this book has applied to automations (Chapter 63) and models (Chapter 64), now applied to cost.
-

Tools

- **Python 3.13 or 3.14** with `pandas` for cost modelling (run here on Python 3.12, `pandas 3.0.2`) — the same tool used throughout this book for every other kind of analysis.
 - **Cloud cost tools**: AWS Cost Explorer and Cost and Usage Reports (or the equivalent on any cloud provider) for real tagging and showback at scale; the AWS Pricing Calculator for estimating a new system's cost before building it.
 - **Pricing sources used in this chapter**, checked in 2026: AWS RDS pricing for `db.m5.large` PostgreSQL and `gp3` storage in `ap-south-1` (Mumbai), checked against multiple independent sources in August 2026, confirming a roughly 42% regional premium over `us-east-1` for identical instance specifications; S3 Standard and Glacier Deep Archive per-GB rates; data transfer-out pricing. LLM API prices reuse Part VI, Chapter 54's own September 2026 figures, so this chapter's numbers stay consistent with the rest of the book rather than introducing a second, conflicting price list.
 - **Companion files (`companion/ch65/`):**
 - `build_ch65_files.py` : builds the full cost model from documented unit prices and Riverstone's real usage volumes.
 - `monthly_cost_model.csv` : all ten platform cost components, in USD and INR.
 - `unit_economics.csv` : cost per report, per prediction, per AI request, and the corrected dual-metric NFR.
-

The project: build and defend a cost model

Goal: produce a real, bottom-up monthly cost model for a system you're responsible for, with unit economics and at least one cost-aware recommendation.

Option A: your own system. Any platform or pipeline with a real, checkable cloud bill.

Option B: Riverstone. Extend `monthly_cost_model.csv` with a plausible ninth or tenth component (a new dashboard tool, a second environment for staging) and recompute the totals.

Steps

1. **List every component** that costs money — compute, storage, requests, transfer — the same inventory discipline as Chapter 63's automation audit, applied to infrastructure.
2. **Find the real unit price for each**, from the provider's current pricing page, not memory — cloud pricing changes, and last year's number is not this year's bill.
3. **Compute the total monthly cost**, and rank components by share, the way section 65.2 did.
4. **Compute at least three unit-economics figures** relevant to what the system actually does (cost per report, per prediction, per transaction).
5. **If there's an existing cost-related NFR or budget**, check it against your bottom-up model. Does it hold up? If not, diagnose why, the way this chapter's story did, rather than just replacing the number.
6. **Propose one cost-aware architecture change** (storage tiering, reserved pricing, a smaller model tier) with an estimated saving.
7. **Sketch a tagging scheme** that would let this system's cost be attributed honestly to whoever owns each part of it.

What good looks like: the total is built bottom-up from real unit prices, not estimated top-down from the current bill; at least one unit-economics figure is genuinely useful for a real future decision; any existing cost target is checked, not assumed correct.

Stretch goals

- Model the cost difference between on-demand and reserved pricing for your most stable, always-on component, using real current rates.
- Design a budget alert for your system, and check — honestly — whether it would fire immediately, the way Riverstone's original NFR would have.
- Compare your system's AI/LLM cost against its human-time cost for the same task, the way section 65.3 compared PO-intake's ₹0.62 API cost against its ₹198 review-time cost.

You've got it when...

- ☐ You can explain how cloud billing works — the meters, on-demand versus reserved — well enough to estimate a new system's cost before building it.
- ☐ You can build a bottom-up cost model for a real platform and identify its largest cost drivers, without assuming they're the newest or most talked-about component.
- ☐ You compute unit economics that are actually useful for a decision, not just a total that sits in a report.
- ☐ You check an existing cost target against a real model rather than trusting it because it's already written down.
- ☐ You can tell the difference between average and marginal cost, and specify which one any cost target actually means.

- ☐ You tag resources at creation and use showback before reaching for chargeback.
 - ☐ You attach a rough cost estimate to architecture decisions before they're made, not after the bill explains them.
-

Recap

- **Cloud billing is pay-as-you-go**, metered by compute-hours, storage-gigabyte-months, requests, and data transfer, with reserved pricing available once a workload is stable enough to commit to.
 - **The warehouse, not the AI, dominates most real data-platform bills**: Riverstone's RDS compute and storage are 66% of spend; its two LLM-powered systems combined are under 2%.
 - **Storage tiering is the highest-leverage lever available**, often a 10x saving for a lifecycle policy that costs nothing further in engineering effort.
 - **Unit economics turn a total into a decision-ready number**: ₹0.62 per PO-intake email, set against ₹198 in human review time, shows exactly where the real cost — and the real opportunity — actually sits.
 - **A cost NFR needs to specify average versus marginal cost explicitly**, or it's not really a checkable number — Chapter 60's original ₹0.50 target, off by roughly 4,400x on a fully-loaded basis, turned out to be a reasonable marginal-cost figure wearing the wrong label.
 - **Tagging at creation, showback before chargeback, and budget alerts that would actually fire meaningfully** are what turn a shared, anonymous bill into an honest, actionable one.
 - **Every architectural choice in this Part has a cost dimension**: storage tiering, reserved compute, scheduled versus event-driven, model tier selection — each worth a rough estimate attached to its ADR, before the decision, not after.
-

Practice exercises

Use `companion/ch65/monthly_cost_model.csv` and `unit_economics.csv`.

Warm-up

1. Name the four meters that drive most cloud bills, and give a one-sentence example of each.
2. What's the difference between on-demand and reserved pricing, and what does an organization give up to get the reserved discount?
3. What's the difference between showback and chargeback, and why does showback usually come first?
4. In your own words, explain the difference between average (fully-loaded) cost and marginal cost, with a non-cloud example.
5. Why might tagging resources at creation be far easier than tagging them a year later?

Core

6. Reproduce section 65.2's cost breakdown. What are the top three cost components, and what share of the total do they represent together?
7. Riverstone's sensor archive splits into a hot and cold tier. Compute the cost per GB for each tier, and explain why the ratio between them matters more than either number alone.
8. Explain, using this chapter's numbers, why "the AI is expensive" would be the wrong conclusion for anyone reviewing Riverstone's platform bill.
9. Chapter 60's original NFR was off by roughly 4,400x. Walk through the diagnosis: what likely caused the gap, and how was the corrected NFR different from simply replacing the number?
10. Compare the cost per PO-intake email (₹0.62) against the ₹198/day assisted-mode labor cost from Chapter 63 (at roughly 40 emails/day, so about ₹5/email in labor). What does this comparison tell you about where a future cost-reduction effort should focus?
11. Design a showback breakdown for a hypothetical fourth team at Riverstone (say, a new "customer experience" team using the support assistant). What would you need to tag, and what share of the current bill might reasonably move to them?
12. A colleague proposes switching Riverstone's PO-intake extraction to the most capable available model "to improve accuracy." Using section 65.5 and Chapter 54's pricing, what would you ask before agreeing?

Stretch

13. Model the reserved-pricing saving for Riverstone's RDS instance and defect-model serving instance combined (₹22,674/month on-demand), assuming a 50% reserved discount. What's the annual saving, and what would you need to be confident about before committing to it?
14. Using `build_ch65_files.py` as a template, add a tenth cost component (a new dashboard tool, additional storage) and recompute the total and the corrected NFR's implied budget.
15. Design a budget alert for Riverstone's platform that would NOT have fired immediately (unlike the original NFR), using this chapter's real total as your baseline.

Think about it

16. Is it ever right to accept a much higher cost than an original target, once you understand why the target was wrong, without treating it as a failure? What would you want documented before doing so?
17. A team consistently shows the highest cost in showback and feels unfairly singled out. How would you investigate whether that's a real signal or an artifact of how shared costs are allocated?

Key terms

FinOps · cloud billing · on-demand pricing · reserved pricing · compute meter · storage meter · request/API meter · data transfer meter · storage tiering · lifecycle policy · unit economics · fully-loaded cost · marginal cost · tagging · showback · chargeback · budget alert · cost-aware architecture

decision

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 60, Designing Whole Systems:** the NFR this chapter checks, corrects, and documents — the same design document, updated a second time.
 - **Chapter 54 (Part VI):** the LLM pricing table this chapter's AI-cost figures are computed from, kept consistent across both chapters.
 - **Chapter 63, Automation Architecture & Governance:** the ownership and inventory discipline this chapter's tagging scheme directly extends to cost.
 - **Chapter 66, Data Strategy, Maturity & Building Data Teams:** the business case for platform investment, now backed by a real cost model rather than an estimate.
 - **Interview preparation:** the Architecture & Leadership Question Bank asks about cost-aware design directly — "how would you estimate what a proposed system will cost, and how would you check that estimate later" is this chapter's method.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. Compute (billed per hour/second a resource runs — a database instance running continuously); storage (billed per GB-month — a data lake's total stored volume); requests/API calls (billed per call — an LLM API charging per token processed); data transfer (billed per GB leaving the provider's network — serving a dashboard's data to users outside the cloud).
2. On-demand charges the full metered rate with no commitment, cancellable anytime. Reserved pricing commits to a usage level for one or three years in exchange for a substantial discount (often 30-70%) — the organization gives up the flexibility to simply stop paying if the workload disappears or shrinks.
3. Showback reports a team's actual cost back to them without charging it against their budget; chargeback actually bills their cost centre for it. Showback comes first because it builds trust in the numbers and gives teams a chance to act on what they see before facing a financial consequence for costs they may not have understood or agreed to.
4. Fully-loaded cost includes all the fixed infrastructure whether or not more work is done — like the total cost of running a restaurant's kitchen for a month, rent and staff included, divided by meals served. Marginal cost is what one more unit costs, given the fixed infrastructure already exists — the cost of the ingredients for one more meal, once the kitchen is already open and staffed for the night.

5. Tagging at creation is a single, automatic step built into how a resource gets provisioned. Tagging a year later means someone has to inventory every existing resource, determine its owner after the fact (potentially without documentation), and apply tags retroactively — exactly the kind of shadow-IT discovery problem Chapter 63's automation audit describes, applied to infrastructure instead of automations.
6. RDS compute (₹16,068), RDS storage (₹9,118), and the tied pair of defect-model serving and orchestration compute (₹3,303 each) are the largest components; RDS compute and storage alone are about 66% of the ₹38,014 total.
7. Hot tier: $₹2,401 \div 1,200 \text{ GB} \approx ₹2.00/\text{GB}$. Cold tier: $₹1,705 \div 9,800 \text{ GB} \approx ₹0.17/\text{GB}$ — roughly a 12x difference per gigabyte. The ratio matters more than either absolute number because it's what tells you the lifecycle policy is doing real work: moving data to the cold tier as it ages is capturing most of that 12x saving on the 89% of the archive that's no longer frequently queried.
8. The two LLM-powered systems (PO-intake extraction and the RAG assistant) combine for under 2% of the total monthly bill, while the always-on warehouse alone is 66%. A reviewer focused on "the AI is expensive" would be scrutinizing the smallest meaningful line item on the invoice while the largest one — the database everyone assumes is just infrastructure — goes unexamined.
9. The original NFR was likely written with marginal cost in mind (what one more 1,000 order lines costs, given the platform already exists) but stated as if it were the fully-loaded figure (which includes the always-on warehouse and orchestration infrastructure regardless of volume). The corrected NFR doesn't just replace the wrong number with the right one — it states both metrics explicitly and labels which is which, so the ambiguity that caused the original error can't recur.
10. With per-email labor cost around ₹5 ($₹198 \div \sim 40 \text{ emails}$) against ₹0.62 in API cost, labor is roughly eight times the infrastructure cost. Any future effort to reduce PO-intake's total cost should focus on reducing review time (better extraction confidence, a faster confirmation UI) rather than further optimizing an already-negligible API bill — exactly the finding section 65.3 draws out explicitly.
11. Tag the support-assistant infrastructure (or the relevant share of its LLM API calls) with the new team's identifier going forward; a reasonable starting share might move the RAG assistant's roughly 0.4% of current spend to the new team, though the real number depends on how usage is actually attributed once the new team's specific queries can be distinguished from existing ones.
12. Ask what specific accuracy problem the current model tier is actually causing (a named failure rate, not a general worry), and what the price difference is per Chapter 54's table — often 5-10x for a materially more capable model. If there's no measured accuracy problem, the proposal is optimizing a cost that was already working, for a benefit nobody has demonstrated is needed.
13. At a 50% reserved discount, ₹22,674/month becomes roughly ₹11,337/month, an annual saving of about ₹1.36 lakh. Before committing, you'd want confidence that both workloads (the warehouse and the defect-model service) will keep running at roughly this scale for the full commitment period — reserving capacity for a workload that might be redesigned or retired within the year converts a savings opportunity into a sunk cost.

- 14.** Personal/computational exercise using the provided script as a template; check that the new component uses a real, sourced unit price and that the total, per-line-item shares, and the NFR's implied per-1,000-line figure are all recomputed consistently from the updated data, not just re-typed.
- 15.** For example, a budget alert set at ₹45,000/month (roughly 18% above the current ₹38,014 baseline) would not fire immediately, giving genuine warning room before an actual overrun rather than firing constantly on entirely normal spend — the difference between a budget alert that teaches people to trust it and one that teaches them to ignore it, the same lesson Chapter 20 taught about alert thresholds generally.
- 16.** Yes — when the higher cost is genuinely justified by what was learned (as in this chapter's story, where ₹2,182.58 turned out to be an entirely reasonable fully-loaded figure once properly understood), rather than an excuse to stop scrutinizing spend. What should be documented: the original target, why it was wrong, what the corrected target is and how it's defined, and what would trigger revisiting it again — exactly the change-log entry this chapter's story added to Riverstone's design document.
- 17.** Check whether the team's apparent cost includes genuinely shared infrastructure (backups, transfer, monitoring) that's been attributed entirely to them rather than split honestly across everyone who benefits from it — Figure 65.3's "shared overhead" category exists precisely to prevent this. If the shared-cost allocation is fair and the team's number is still highest, investigate whether that reflects real, justified usage (a genuinely heavier workload) before assuming either the team or the accounting is at fault.